3.11 GitHub Actions: Workflows, Jobs and Triggers

Topic 11 of 13 | ~4 hours | Difficulty: Intermediate

1 What it is

GitHub Actions is the automation and CI/CD (continuous integration / continuous delivery) service built into GitHub. You describe automation in YAML files in `.github/workflows/`. Each file is a **workflow**. The vocabulary is small but precise:

- **Event / trigger:** something that happens (a push, a pull request, a tag, a schedule, a manual button) and starts a workflow run.
- **Workflow:** one YAML file; it can have several triggers and several jobs.
- **Job:** a group of steps that runs on **one runner** (a fresh virtual machine or container host). Jobs run **in parallel** by default; `needs:` makes them sequential.
- **Step:** one unit inside a job, either a shell command (`run:`) or a reusable **action** (`uses: owner/repo@version`). Steps share the job's filesystem and run in order.
- **Runner:** the machine executing a job. `runs-on: ubuntu-latest` picks a GitHub-hosted Linux VM. (See artifacts and runners §3.13.)
- **Matrix:** one job definition expanded into many runs (for example Node 20 and 22 on two operating systems).
- **Cache:** saved files (like `~/.npm`) restored on later runs so dependency installs are fast.

Because every job starts on a clean machine, nothing carries over between jobs unless you pass **outputs** or upload **artifacts**.

2 Why it matters in DevOps

- **It is the pipeline.** Everything from CI/CD §0.2 (build, test, scan, publish, deploy) is a workflow living next to the code, reviewed like code.
- **Required checks** on protected branches §3.8 are workflow jobs. A flaky or slow workflow blocks the whole team.

- **Cost and speed:** minutes on private repositories are billed (public repositories get standard hosted runners free). Caching, concurrency cancellation and sensible triggers directly cut cost and wait time.
- **Security:** a workflow can read secrets and push to your registry. Over-broad `permissions`, untrusted pull-request input, and unpinned third-party actions are real attack paths (see secrets and environments §3.12).
- **Reproducibility:** "works on CI" is the standard; if the pipeline is the only way to ship, it must be deterministic.

3 Real-world system example

Open-source projects on GitHub. Many open-source libraries test every PR across several operating systems and language versions using a matrix; the `actions/setup-node` action, maintained by GitHub, documents exactly this matrix pattern in its README. Most popular repositories show a CI status badge driven by a workflow file in `.github/workflows/`.

The `actions/*` and Marketplace ecosystem. GitHub maintains first-party actions (`actions/checkout`, `actions/cache`, `actions/upload-artifact`, `actions/setup-node`) and the Marketplace hosts thousands of community actions, including `docker/build-push-action`, `hashicorp/setup-terraform` and `aws-actions/configure-aws-credentials`. A typical production service chains them: checkout, setup language, restore cache, test, build image, push to a registry, deploy.

Supply-chain incidents such as the 2025 compromise of a widely used third-party GitHub Action (`tj-actions/changed-files`) that exposed secrets in workflow logs are the reason security guidance says to pin third-party actions to a full commit SHA and to keep `permissions` minimal.

4 Diagram

Anatomy of a run:

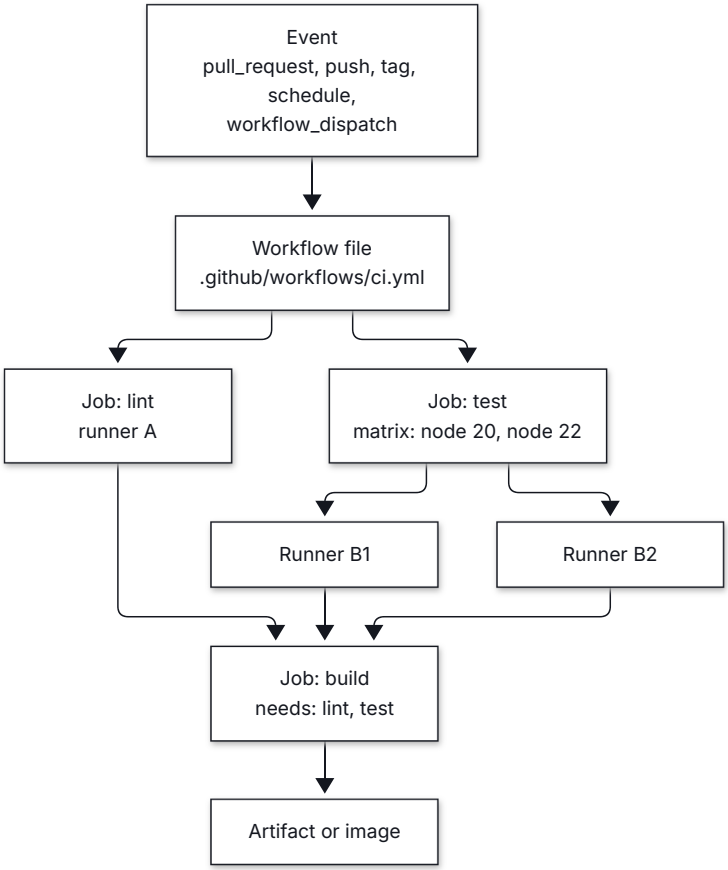


Figure 3.11-1

What happens inside one job:

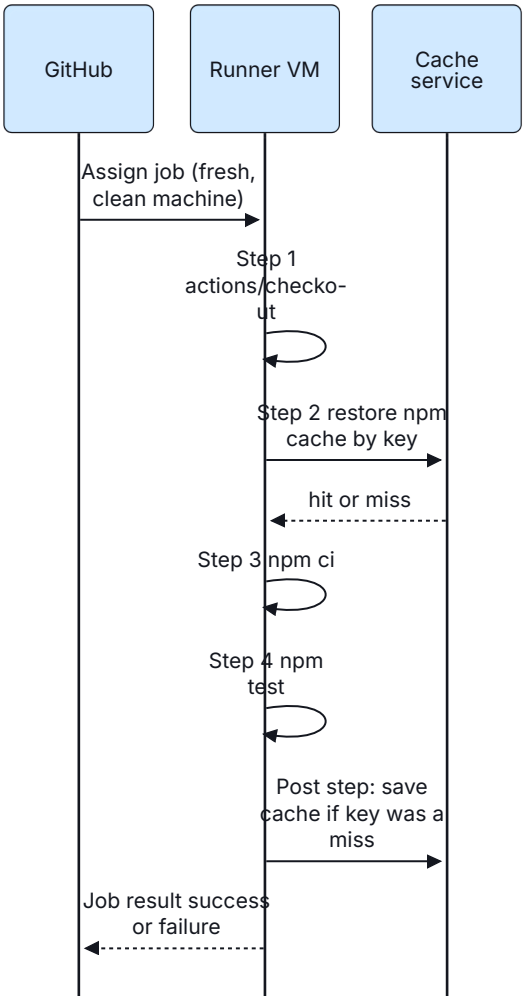


Figure 3.11-2

5 How it works

Common triggers (`on:`)

Event	Fires when	Notes
`push`	commits or tags are pushed	filter with `branches:` , `tags:` , `paths:`
`pull_request`	PR opened, synchronized (new commits), reopened	runs on the merge of the PR into the base; for forks, secrets are not available and the token is read-only

Event	Fires when	Notes
`workflow_dispatch`	manual run from the UI/CLI/API	can define `inputs:`
`schedule`	cron (UTC), e.g. `'17 3 * * 1-5'`	minimum 5 minutes; may be delayed at busy times; runs on the default branch
`workflow_call`	another workflow calls this one (reusable workflow)	defines `inputs`, `secrets`, `outputs`
`release` / tag `push`	a release is published / a tag is pushed	release pipelines (semver §3.10)
`merge_group`	merge queue builds a candidate	required for merge queue

Key workflow syntax

Key	Purpose
`name`	display name
`on`	triggers and filters
`permissions`	scopes of the automatic `GITHUB_TOKEN`; set top-level `contents: read` and raise per job only when needed
`env`	environment variables at workflow, job or step level
`concurrency`	`group:` + `cancel-in-progress: true` cancels superseded runs
`jobs.<id>.runs-on`	runner label
`jobs.<id>.needs`	dependency on other jobs
`jobs.<id>.if`	conditional, e.g. `if: github.ref == 'refs/heads/main'`
`jobs.<id>.timeout-minutes`	kill hung jobs (default is 360, far too long)
`jobs.<id>.strategy.matrix`	combinations; `fail-fast`, `max-parallel`, `include`, `exclude`
`jobs.<id>.services`	sidecar containers such as PostgreSQL and Redis
`jobs.<id>.outputs`	values passed to later jobs
`steps[*].uses` / `run` / `with` / `env` / `id` / `if`	the work itself

Expressions and contexts. `${{ ... }}` evaluates expressions over contexts: `github` (event, ref, sha, actor), `env`, `secrets`, `matrix`, `needs`, `steps`, `runner`, `vars`. `github.ref_name` is the branch or tag name; `github.sha` the commit. Never interpolate untrusted values (PR titles, branch names) directly into a `run:` script; pass them through `env:` and quote the variable.

Caching. `actions/cache` stores a directory under a **key**; if the exact key exists it is restored (hit), otherwise `restore-keys` provide a prefix fallback and the cache is saved after the job. Most `setup-*` actions have a built-in `cache:` input (`cache: npm`) that keys on your lockfile. Caches are per-repo, scoped by branch (a PR can read the base branch's cache but not the reverse), evicted after about 7 days without access, and limited in total size per repository (10 GB at the time of writing; check the docs). Caches speed things up, artifacts pass results: do not confuse them.

Pinning. `uses: actions/checkout@v4` follows a moving major-version tag. For third-party actions prefer a full commit SHA with a comment (`uses: owner/action@<40-char-sha> # v2.3.1`); Dependabot can keep those updated. Check the Marketplace or each action's releases page for the current major version when you write new workflows.

6 Step-by-step guide

Prerequisites: a GitHub account, Git, and Node.js 20+ locally for the sample app. Public repositories run on hosted runners at no cost; for private repositories watch your included minutes.

1. **Create a tiny `shop-api` project with a passing test.**

```bash
mkdir shop-api-actions && cd shop-api-actions && git init -b main
cat > package.json <<'EOF'
{
  "name": "shop-api",
  "version": "0.1.0",
  "private": true,
  "scripts": {
    "lint": "node --check src/index.js",
    "test": "node --test"
  }
}
EOF
mkdir -p src test
cat > src/index.js <<'EOF'
function total(items) {
  return items.reduce((sum, i) => sum + i.price * i.qty, 0);
}
module.exports = { total };
EOF
cat > test/total.test.js <<'EOF'
const test = require('node:test');
const assert = require('node:assert');
const { total } = require('../src/index.js');
test('adds up the cart', () => {
  assert.strictEqual(total([{ price: 10, qty: 2 }, { price: 5, qty: 1 }]), 25);
```

```
});
EOF
npm install --package-lock-only >/dev/null && npm test
```

Expected: the TAP output ends with `# pass 1` and `# fail 0`. The generated `package-lock.json` is what `npm ci` and the cache key use.

2. **Write your first workflow.** `.github/workflows/ci.yml`:

```yaml
name: CI

on:
  pull_request:
  push:
    branches: [main]
  workflow_dispatch:

permissions:
  contents: read

concurrency:
  group: ci-${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true

jobs:
  lint:
    runs-on: ubuntu-latest
    timeout-minutes: 5
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm
      - run: npm ci
      - run: npm run lint

  test:
    runs-on: ubuntu-latest
    timeout-minutes: 10
    strategy:
      fail-fast: false
      matrix:
        node: [20, 22]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: ${{ matrix.node }}
          cache: npm
```

```
      - run: npm ci
      - run: npm test

  build:
    needs: [lint, test]
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: echo "building ${GITHUB_SHA::7} on ${GITHUB_REF_NAME}"
```

The job id `test` is what you would mark as a required check (matrix jobs appear as `test (20)` and `test (22)`).

3. **Push and watch it run.**

```bash
git add . && git commit -m "ci: add workflow"
gh repo create shop-api-actions --public --source . --push
gh run watch
gh run list --limit 3
```

In the web UI open *Actions*, then the run: you see `lint`, two `test` matrix jobs in parallel, and `build` only after they finish.

4. **See caching work.** Re-run the workflow (`gh run rerun <run-id>`, or push an empty commit with `git commit --allow-empty -m "ci: retry" && git push`). In the `setup-node` step log, the first run says the cache was not found; the second says `Cache restored from key: node-cache-Linux-x64-npm-<hash>`. Compare `npm ci` durations.

5. **Add a service container and an environment variable.** Add this under the `test` job (above `steps:`) to run tests against a real PostgreSQL:

```yaml
    env:
      DATABASE_URL: postgres://postgres:postgres@localhost:5432/shop
    services:
      postgres:
        image: postgres:16
        env:
          POSTGRES_PASSWORD: postgres
          POSTGRES_DB: shop
        ports:
          - 5432:5432
        options: >-
          --health-cmd "pg_isready -U postgres"
          --health-interval 5s
          --health-timeout 5s
          --health-retries 10
```

The health options make the job wait until the database is ready before steps start. Add a step `- run: pg_isready -h localhost -p 5432` to prove it is reachable.

6. **Use path filters and conditions.** Skip heavy jobs for documentation-only changes, and run a deploy-looking step only on `main`:

```yaml
on:
  pull_request:
    paths-ignore: ['**.md', 'docs/**']
```

```yaml
      - name: Only on main
        if: github.ref == 'refs/heads/main' && github.event_name == 'push'
        run: echo "would deploy here"
```

If you make this workflow a required check, remember the skipped-check trap from topic 8 §3.8.

7. **Pass data between jobs with outputs.**

```yaml
  version:
    runs-on: ubuntu-latest
    outputs:
      short_sha: ${{ steps.vars.outputs.short_sha }}
    steps:
      - id: vars
        run: echo "short_sha=${GITHUB_SHA::7}" >> "$GITHUB_OUTPUT"
  use-it:
    needs: version
    runs-on: ubuntu-latest
    steps:
      - run: echo "image tag will be shop-api:${{ needs.version.outputs.short_sha }}"
```

8. **Trigger on tags and on a schedule.**

```yaml
on:
  push:
    tags: ['v*.*.*']
  schedule:
    - cron: '17 3 * * 1-5'      # 03:17 UTC on weekdays
```

Push `git tag -a v0.1.0 -m "Release 0.1.0" && git push origin v0.1.0` and confirm a run starts, with `GITHUB_REF_NAME` equal to `v0.1.0`.

9. **Run it locally for fast feedback (optional).** The community tool `act` runs many workflows in Docker; it is approximate (not identical to GitHub-hosted runners), so treat the real run as the truth.

7 Real-world scenario

(Illustrative.) The `shop-api` pipeline takes 14 minutes because every job reinstalls dependencies and a dev pushes five fixup commits to a PR, queueing five full runs. Costs on the private repo's minutes climb and reviewers wait. The platform engineer adds `cache: npm`, a `concurrency` group with `cancel-in-progress: true` for PR refs, a path filter that skips the docs-only PRs, and splits lint from tests so failures surface in two minutes. Pipeline time drops to about 4 minutes. Later, a `pull_request_target` workflow someone copied from a blog post checks out the PR head with write permissions; a review catches it before a fork can exfiltrate a token, and the team replaces it with a plain `pull_request` workflow. **Lesson:** a workflow is production code. Optimise for feedback time, cost, and least privilege together.

8 Common mistakes and best practices

Don't

- Use `pull_request_target` or `workflow_run` to run untrusted PR code with secrets.
- Interpolate `${{ github.event.pull_request.title }}` (or any user-controlled text) straight into a `run:` script: script injection. Pass it via `env:`.
- Leave default permissions broad; a compromised step then owns your repository.
- Use `@main` or `@latest` for third-party actions, or trust a mutable tag without a review.
- Rely on a cache for correctness. It can miss or be evicted at any time.
- Let jobs hang for the default 6 hours: always set `timeout-minutes`.

Do

- Set `permissions: contents: read` at the top and grant more per job.
- Add `concurrency` with `cancel-in-progress` on PR workflows (not on deploy workflows that must finish).
- Use the lockfile-based `npm ci`, not `npm install`, in CI.
- Pin third-party actions to full SHAs and let Dependabot update them.
- Keep logic in scripts or `make` targets that you can run locally, with the workflow as thin glue.
- Use reusable workflows (`workflow_call`) once three repositories share the same pipeline.

9 Tasks

- [ ] **T1 — First green run** *(≈30 min, beginner)*
  - **Goal:** get a workflow running on every PR.
  - **Steps:**
    1. Build the project and the `ci.yml` from guide steps 1-3.
    2. Open a PR that changes `src/index.js` and watch the checks.
  - **Done when:** the PR shows `lint`, `test (20)`, `test (22)` and `build` all green, and you can point to the line in the log where `npm test` printed `# pass 1`.

- [ ] **T2 — Extend the matrix** *(≈25 min, beginner)*
  - **Goal:** control the matrix.
  - **Steps:**
    1. Add `os: [ubuntu-latest, windows-latest]` and set `runs-on: ${{ matrix.os }}`.
    2. Use `exclude` to drop Node 20 on Windows.
    3. Use `include` to add one extra combination that is not in the grid (for example `node: 18` on `ubuntu-latest`) with a variable `experimental: true`.
  - **Done when:** the run lists exactly the expected number of matrix jobs (4 in the grid minus 1 excluded, plus 1 from `include`, so 4 `test` jobs), shown in the run summary.

- [ ] **T3 — Measure the cache** *(≈25 min, intermediate)*
  - **Goal:** prove caching changes build time.
  - **Steps:**
    1. Run the workflow twice with `cache: npm`; note `npm ci` step time each run.
    2. Remove `cache: npm`, run again, and compare.
    3. Change `package-lock.json` slightly and observe the cache miss and re-save.
  - **Done when:** you have a table of step durations and the log lines for "Cache restored" versus "Cache not found".

- [ ] **T4 — Break it: script injection** *(≈35 min, advanced)*
  - **Goal:** understand why untrusted input must not reach `run:`.
  - **Steps:**
    1. In a throwaway repo, add a step `run: echo "Title: ${{ github.event.pull_request.title }}"`.
    2. Open a PR titled `x"; echo INJECTED; echo "` and read the log.
    3. Fix it with `env: TITLE: ${{ github.event.pull_request.title }}` and `run: echo "Title: $TITLE"`.

- **Done when:** the first version prints `INJECTED` as its own command output and the fixed version prints the title literally.

☐ **T5 — Cost control** *(≈30 min, intermediate)*

- **Goal:** stop wasted runs.
- **Steps:**
  1. Add a `concurrency` group with `cancel-in-progress: true` and `timeout-minutes` on every job.
  2. Push three commits quickly to a PR branch.
  3. Add `paths-ignore` for Markdown and push a docs-only change.
- **Done when:** the Actions tab shows earlier runs as "Cancelled" and the docs-only push triggers no run.

☐ **T6 — Prove it: tag-triggered release skeleton** *(≈45 min, advanced)*

- **Goal:** build the shape of the capstone pipeline from memory.
- **Steps:**
  1. Without the guide, write one workflow that runs lint and tests on PRs and, on a `v*.*.*` tag, runs the tests again and then a `release` job that `needs: test`.
  2. Give the release job `contents: write` only.
  3. Explain in writing what happens when tests fail on a tag.
- **Done when:** pushing a PR runs only the checks, pushing `v0.2.0` also runs the `release` job, and a deliberately failing test prevents it.

10 Self-check questions

1. Do jobs in a workflow share a filesystem? How do you pass data between them?
2. What is the difference between a cache and an artifact?
3. How do you make job B wait for job A, and what happens to B if A fails?
4. Why is `${{ github.event.issue.title }}` inside a `run:` script dangerous?
5. What does `concurrency: cancel-in-progress` do, and when should you not use it?
6. How does a matrix job appear as a required status check?

ANSWERS

1. No, each job gets its own runner. Use job `outputs` for small values and artifacts (or a registry) for files.
2. A cache speeds up repeated work and may be missing; an artifact is a deliberate output of a run that you download or hand to a later job.

3. `needs: [A]`. If A fails, B is skipped unless it uses `if: always()` or a similar condition.
4. The text is substituted into the script before the shell runs it, so an attacker-controlled title can inject commands. Pass it through `env:`.
5. It cancels older runs in the same group when a new one starts. Avoid it for deployments or releases that must complete once started.
6. Each combination is its own check named like `test (20)`; make a final aggregating job (for example `ci-ok` with `needs:` on all and `if: always()`) the single required check.

11 Further reading

- GitHub Docs, "Workflow syntax for GitHub Actions", "Events that trigger workflows", "Using jobs in a workflow", "Dependency caching reference" and "Security hardening for GitHub Actions" (docs.github.com/actions).
- `actions/checkout`, `actions/setup-node`, `actions/cache` repositories on GitHub (README files show current usage and versions).
- *Continuous Delivery* by Jez Humble and David Farley.
- See also Level 5: GitHub Actions for CI/CD §5.2 for deployment-focused pipelines.